BAN CƠ YẾU CHÍNH PHỦ
HỌC VIỆN KỸ THUẬT MẬT MÃ



BÁO CÁO ĐỀ TÀI
XÂY DỰNG ỨNG DỤNG "SOS & AN TOÀN GIA ĐÌNH" TÍCH HỢP CƠ CHẾ BẢO MẬT

Môn học: An toàn và bảo mật trong phát triển PM di động
Lớp: L03

Sinh viên thực hiện:
Phạm Văn Tùng
Phạm Nguyên Vũ
Đỗ Văn Tuấn

Người hướng dẫn:
GV: Thái Thị Thanh Vân

Khoa Công nghệ thông tin – Học viện Kỹ thuật Mật Mã
Hà Nội, 2026

MỤC LỤC

DANH SÁCH HÌNH ẢNH **4**
LỜI NÓI ĐẦU **5**
CHƯƠNG I. CƠ SỞ LÝ THUYẾT **6**
I.1. Tổng quan về an toàn và bảo mật trên thiết bị di động 6
I.2. Một số phương pháp tăng cường bảo mật ứng dụng di động 6
I.2.1. Bảo mật mã nguồn và chống dịch ngược (ProGuard / R8) 6
I.2.2. Bảo mật tệp tin và cơ sở dữ liệu (EncryptedPrefs & SQLCipher) 7
I.2.3. Bảo mật phương thức kết nối và kênh truyền 8
I.2.4. Thực hiện xác thực đầu vào và chống giả mạo GPS 9
I.2.5. Sử dụng mật mã an toàn: Curve25519, HKDF, AES-256-GCM 10
I.2.6. Thực hiện kiểm tra thâm nhập (Penetration Testing) 12
I.3. Cơ chế bảo mật thường được sử dụng trên các thiết bị di động 13
I.3.1. Xác thực sinh trắc học trên Android (BiometricPrompt) 13
I.3.2. Hệ thống kho khóa Android (Android Keystore System) 15
I.3.3. Bảo vệ giao diện và chống quay chụp màn hình (FLAG_SECURE) 17
I.3.4. Phòng thủ chủ động: Phát hiện Root và kiểm tra tính toàn vẹn 18
I.3.5. Xác thực Ngoài băng (OOB) bằng SHA-256 Key Fingerprint 19
CHƯƠNG II. TỔNG QUAN VỀ ỨNG DỤNG VÀ THIẾT KẾ HỆ THỐNG **20**
II.1. Giới thiệu về ứng dụng 20
II.1.1. Tính cấp thiết của đề tài 20
II.1.2. Mục tiêu nghiên cứu 20
II.1.3. Nội dung và phạm vi đề tài 21
II.2. Khảo sát và Phân tích bài toán 21
II.2.1. Môi trường hoạt động và công nghệ sử dụng 21
II.2.2. Xác định mục tiêu và ràng buộc của bài toán 23
II.3. Thiết kế kiến trúc hệ thống 24
II.3.1. Kiến trúc 3 tầng và Ranh giới tin cậy (Trust Boundary) 24
II.3.2. Kiến trúc Emergency Bypass (Triết lý Zero Barrier) 26
II.3.3. Thiết kế mô hình dữ liệu 27
II.4. Ma trận phân tích đe dọa theo mô hình STRIDE 28
CHƯƠNG III: TÍCH HỢP CÁC CƠ CHẾ BẢO MẬT VÀ KẾT QUẢ THỰC NGHIỆM **30**
III.1. Xác thực Google Sign-In và Key Provisioning Keystore tự động 30
III.2. Thỏa thuận khóa Diffie-Hellman trên Curve25519 và HKDF-SHA256 31
III.3. Mã hóa đầu cuối (E2EE) dữ liệu vị trí và SOS bằng AES-256-GCM 32
III.4. Cơ chế chống phát lại gói tin cứu hộ (ReplayGuard) 33
III.5. Tích hợp cơ chế xác thực bằng sinh trắc học Scoped Biometrics 34
III.6. Tích hợp cờ bảo mật Native FLAG_SECURE 35
III.7. Tích hợp phát hiện Root và giả mạo vị trí GPS 35
III.8. Đánh giá hiệu năng mật mã (Crypto Benchmark 300 vòng lặp) 36

III.9. Kết quả thực nghiệm 5 kịch bản tấn công bảo mật (ATK-01 -> ATK-05) 37
III.10. Thống kê kết quả kiểm thử tự động 100/100 Test Cases 39
CHƯƠNG IV: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN **40**
IV.1. Kết quả đạt được 40
IV.2. Những hạn chế còn tồn tại 40
IV.3. Hướng phát triển trong tương lai 41
TÀI LIỆU THAM KHẢO **42**
PHỤ LỤC: TỔNG HỢP CÁC CƠ CHẾ BẢO MẬT ĐÃ TRIỂN KHAI **43**

DANH SÁCH HÌNH ẢNH

Figure 1-Kiến trúc 3 tầng và ranh giới tin cậy của ứng dụng 25
Figure 2-Kiến trúc Emergency Bypass: Zero Barrier và Scoped Gate 26
Figure 3-Ma trận phân tích đe dọa theo mô hình STRIDE 29
Figure 4-Giao thức thỏa thuận khóa ECDH X25519 và Key Provisioning M8 31
Figure 5-Luồng mã hóa E2EE và cơ chế kiểm tra chống phát lại 33
Figure 6-Quy trình nhận và kiểm tra gói tin SOS qua ReplayGuard 34

LỜI NÓI ĐẦU

Với sự phát triển rất nhanh của các ứng dụng di động, việc bảo mật và an toàn cho các ứng dụng là một lĩnh vực đặc biệt quan trọng vì nó được truy cập mọi lúc mọi nơi và được sử dụng bởi bất kỳ ai. Tuy vậy, không phải bất kỳ nhà phát triển ứng dụng nào cũng chú tâm đến việc bảo mật an ninh, vì vậy nhiều ứng dụng vẫn tồn tại với các nguy cơ mất an toàn và bảo mật gây ra nhiều thiệt hại cho người sử dụng và các bên liên quan.

Chính vì thế mà nhóm chúng em đã tìm hiểu các cơ chế an toàn và bảo mật để áp dụng vào việc xây dựng ứng dụng "SOS & An toàn Gia đình" làm cho ứng dụng trở nên an toàn và bảo mật hơn với người dùng. Nội dung báo cáo gồm 4 chương:

- **CHƯƠNG I: CƠ SỞ LÝ THUYẾT**
- **CHƯƠNG II: TỔNG QUAN VỀ ỨNG DỤNG VÀ THIẾT KẾ HỆ THỐNG**
- **CHƯƠNG III: TÍCH HỢP CÁC CƠ CHẾ BẢO MẬT VÀ KẾT QUẢ THỰC NGHIỆM**
- **CHƯƠNG IV: TỔNG KẾT VÀ HƯỚNG PHÁT TRIỂN**

Trong quá trình thực hiện đề tài, nhóm chúng em đã cố gắng tìm hiểu tài liệu qua nhiều phương thức để tổng hợp kiến thức, học hỏi từ thầy cô và bạn bè để hoàn thành. Nhóm chúng em xin chân thành cảm ơn cô giáo Thái Thị Thanh Vân cùng các thầy cô trong Khoa Công nghệ Thông tin – Học viện Kỹ thuật Mật mã đã tận tình chỉ dẫn và giúp đỡ chúng em hoàn thành đề tài này.

Trong bài báo cáo còn có thiếu sót mong thầy cô và các bạn góp ý để nhóm em hoàn thiện hơn.

CHƯƠNG 1 — CƠ SỞ LÝ THUYẾT VỀ AN TOÀN VÀ BẢO MẬT DI ĐỘNG

1.1. Tổng quan an toàn thông tin trên nền tảng di động

Môi trường thực thi của ứng dụng di động có những đặc thù rủi ro hoàn toàn khác biệt so với các hệ thống máy chủ doanh nghiệp truyền thống:

1. **Thiết bị hoạt động trong môi trường thù địch (Hostile Environment):** Thiết bị di động dễ bị thất lạc, đánh cắp vật lý, hoặc bị can thiệp bởi chính người dùng thông qua việc Root máy/Jailbreak nhằm vượt qua các cơ chế kiểm soát bảo mật của hệ điều hành.

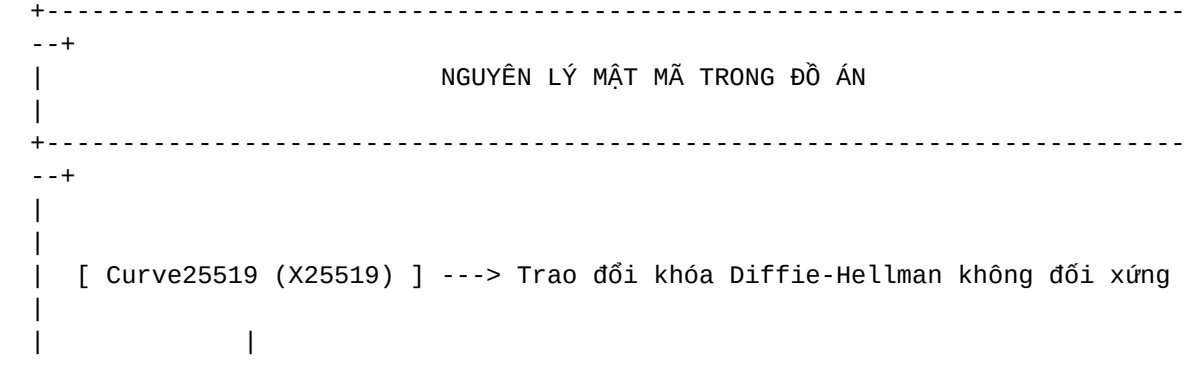
2. **Kênh truyền thông không tin cậy:** Thiết bị thường xuyên kết nối qua các mạng Wi-Fi công cộng không có mật khẩu hoặc mạng di động của nhà mạng, nơi các cuộc tấn công nghe lén (Eavesdropping), bắt gói tin (Sniffing) và giả mạo máy chủ (DNS Spoofing / MITM) luôn tiềm ẩn.

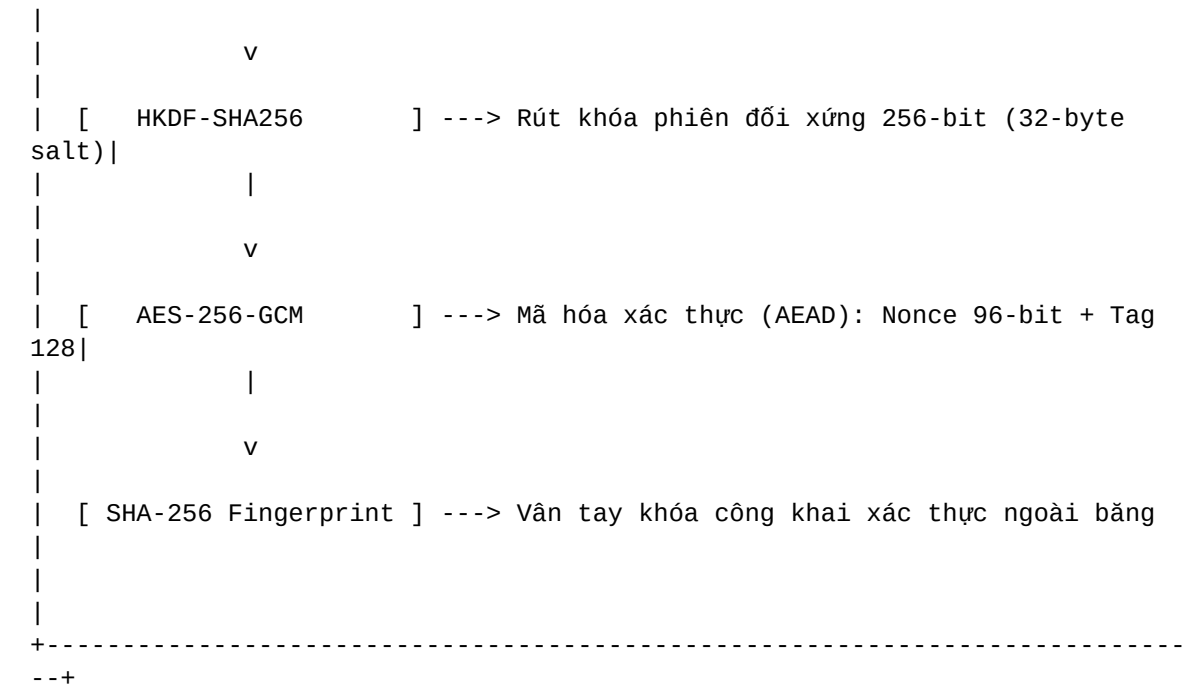
3. **Mô hình Zero Trust đối với máy chủ lưu trữ:** Trong kiến trúc an toàn di động hiện đại (theo chuẩn NIST SP 800-207), máy chủ trung gian không được coi là một thực thể tin cậy tuyệt đối. Nếu máy chủ nắm giữ khóa giải mã hoặc dữ liệu văn bản rõ, một sự cố rò rỉ cơ sở dữ liệu (data breach) hoặc sự can thiệp của quản trị viên nội bộ sẽ làm phá sản toàn bộ tính riêng tư của hệ thống.

Do đó, ba nguyên lý kiến trúc bắt buộc phải được áp dụng:

- **Dữ liệu nhạy cảm phải được mã hóa đầu cuối (E2EE):** Chỉ người gửi và người nhận hợp pháp sở hữu khóa giải mã; máy chủ chỉ làm chức năng định tuyến (blind relay).
- **Khóa bí mật phải được neo giữ trong phần cứng an toàn:** Khóa riêng tư không được phép lưu trữ dưới dạng file văn bản hay SharedPreferences thông thường mà phải được niêm phong trong Android Keystore (TEE/StrongBox).
- **Tính toàn vẹn của thiết bị là tiên quyết:** Ứng dụng phải có khả năng tự kiểm tra trạng thái root và phát hiện vị trí GPS giả lập để đánh giá mức độ tin cậy của dữ liệu đầu vào.

1.2. Các nguyên lý mật mã sử dụng trong đồ án





1.2.1. Thỏa thuận khóa Diffie-Hellman trên Curve25519 (X25519)

Giao thức trao đổi khóa X25519 (RFC 7748) do nhà mật mã học Daniel J. Bernstein phát triển, hoạt động trên đường cong Montgomery được định nghĩa bởi phương trình:

$$y^2 = x^3 + 486662x^2 + x$$

trên trường hữu hạn nguyên tố $\mathbb{F}_p$ với $p = 2^{255} - 19$.

Ưu điểm vượt trội so với các đường cong Weierstrass truyền thống (NIST P-256):

- **Hiệu năng và Tốc độ:** Thuật toán sử dụng phép nhân thang Montgomery (Montgomery ladder) chỉ thao tác trên tọa độ hoành độ x, giúp loại bỏ hoàn toàn nhu cầu tính toán tọa độ tung độ y. Phép toán chỉ bao gồm các phép nhân và cộng trên trường hữu hạn, đạt tốc độ xử lý nhanh hơn từ 2 đến 4 lần so với NIST P-256 trên kiến trúc CPU ARM của di động.
- **Khả năng kháng tấn công kênh kề (Side-Channel Resistance):** Thuật toán chạy trong thời gian hằng số (constant-time execution), không có các rẽ nhánh điều kiện phụ thuộc vào giá trị của bit khóa bí mật, miễn nhiễm với các kỹ thuật tấn công đo thời gian (Timing Attack) và phân tích năng lượng (Power Analysis).
- **An toàn xoắn (Twist Security):** Curve25519 được thiết kế có tính an toàn xoắn hai chiều (twist-secure), ngăn chặn triệt để các cuộc tấn công gửi điểm thuộc đường cong xoắn yếu (invalid curve attack) mà không cần bước kiểm tra điểm hợp lệ phức tạp.
- **Loại bỏ nguy cơ Backdoor:** Không giống như các hằng số sinh ngẫu nhiên khó giải thích của chuẩn NIST, các hệ số của Curve25519 được lựa chọn minh bạch dựa trên tiêu chí toán học chặt chẽ.

Cơ chế hoạt động trong đồ án: Mỗi người dùng khi đăng nhập sẽ sinh một cặp khóa X25519 gồm Private Key ($k_{priv} \in [0, 2^{256}-1]$ đã được kẹp bit - clamped) và Public Key ($K_{pub} = k_{priv} \cdot G$). Khi hai thành viên A và B muốn giao tiếp an toàn:

$$\text{SharedSecret} = X25519(k_{priv}^A, K_{pub}^B) = X25519(k_{priv}^B, K_{pub}^A)$$

Bên thứ ba hoặc máy chủ chuyển tiếp dù thu thập được cả K_{pub}^A và K_{pub}^B trên đường truyền cũng không thể tìm được SharedSecret do độ khó tính toán của bài toán Logarithm rời rạc trên đường cong Elliptic (ECDLP).

1.2.2. Hàm rút khóa HKDF-SHA256 (RFC 5869)

Khóa bí mật dùng chung SharedSecret sinh ra từ phép nhân điểm X25519 có độ dài 32 byte nhưng phân phối entropy chưa đồng đều tuyệt đối theo tiêu chuẩn mật mã học. Do đó, việc sử dụng trực tiếp làm khóa mã hóa AES là không an toàn. Đồ án áp dụng chuẩn HKDF (HMAC-based Extract-and-Expand Key Derivation Function) bao gồm 2 bước:

1. **Giai đoạn Trích xuất (Extract):** Tạo khóa giả ngẫu nhiên PRK (Pseudorandom Key) có độ dài 256-bit:

$$\text{PRK} = \text{HMAC-SHA256}(\text{salt}, \text{SharedSecret})$$

2. **Giai đoạn Mở rộng (Expand):** Rút ra khóa đối xứng phiên phục vụ mã hóa:

$$\text{OKM} = \text{HMAC-SHA256}(\text{PRK}, \text{info} \parallel \text{0x01})$$

với $\text{info} = \text{"sos_safe_family_aes_key"}$.

Xử lý lỗi kỹ thuật sâu: Lỗi "Empty Key" trên Android JCE:

Theo lý thuyết RFC 5869, tham số **`salt`** trong bước **`Extract`** có thể để rỗng (chiều dài 0). Tuy nhiên, trên hệ điều hành Android, bộ cung cấp mật mã Java Cryptography Extension (JCE) của một số nhà sản xuất thiết bị (như Samsung Knox hoặc Xiaomi MIUI) từ chối khởi tạo đối tượng **`SecretKeySpec`** với mảng byte rỗng và ném ra biệt lệ nghiêm trọng: **`java.security.InvalidKeyException: Empty key`**. Điều này khiến ứng dụng chạy tốt trên môi trường test máy tính nhưng lại bị crash trên thiết bị thật.

Để giải quyết triệt để lỗi này mà vẫn đảm bảo tính tương thích toán học 100% với RFC 5869, mã nguồn của đồ án sử dụng một mảng cố định gồm 32 byte **`0x00`**:

```
_hkdf.deriveKey(
  secretKey: sharedSecret,
  nonce: Uint8List(32), // Salt 32 byte 0x00 — tương đương chuẩn RFC 5869,
khắc phục triệt để lỗi JCE
  info: utf8.encode('sos_safe_family_aes_key'),
);
```

1.2.3. Tiêu chuẩn mã hóa xác thực AES-256-GCM (NIST SP 800-38D)

AES-256-GCM là cơ chế mã hóa có xác thực (Authenticated Encryption with Associated Data - AEAD) kết hợp giữa chế độ đếm CTR của thuật toán AES và hàm băm xác thực vạn năng GHASH trên trường Galois $GF(2^{128})$ với đa thức tối thiểu:

$$P(x) = x^{128} + x^7 + x^2 + x + 1$$

Mỗi lần thực thi mã hóa, thuật toán yêu cầu:

- **Nonce (Number used ONCE):** 96-bit (12 byte), được sinh ngẫu nhiên độc lập bằng bộ sinh số an toàn mật mã CSPRNG.
- **Ciphertext:** Bản mã của dữ liệu vị trí/SOS/tin nhắn.
- **Authentication Tag:** Mã xác thực toàn vẹn 128-bit (16 byte).

Tính chất then chốt: Thao tác giải mã sẽ tự động thất bại và ném ngoại lệ **`SecretBoxAuthenticationError`** nếu có bất kỳ sự can thiệp nào vào bản mã (dù chỉ đổi 1 bit), sửa đổi nonce, hay tráo đổi tag xác thực.

Hiểm họa tái sử dụng Nonce (Nonce Reuse Catastrophe): Trong chế độ GCM, nếu một cặp $(\text{Key}, \text{Nonce})$ bị tái sử dụng cho hai bản rõ khác nhau, kẻ tấn công có thể thực hiện phép XOR hai bản mã để thu được XOR của hai bản rõ ($C_1 \oplus C_2 = P_1 \oplus P_2$), đồng thời suy ra khóa băm GHASH $H = \text{AES}_K(0^{128})$, từ đó giả mạo hoàn toàn mã xác thực của các gói tin tiếp theo. Đồ án ngăn chặn rủi ro này bằng cách: mỗi lần mã hóa vị trí hay phát SOS đều bắt buộc gọi **`Random.secure()`** để sinh nonce 96-bit mới, triệt tiêu xác suất trùng lặp ($1/2^{96}$).

1.2.4. Xác thực Ngoài băng tần (OOB) bằng SHA-256 Key Fingerprint

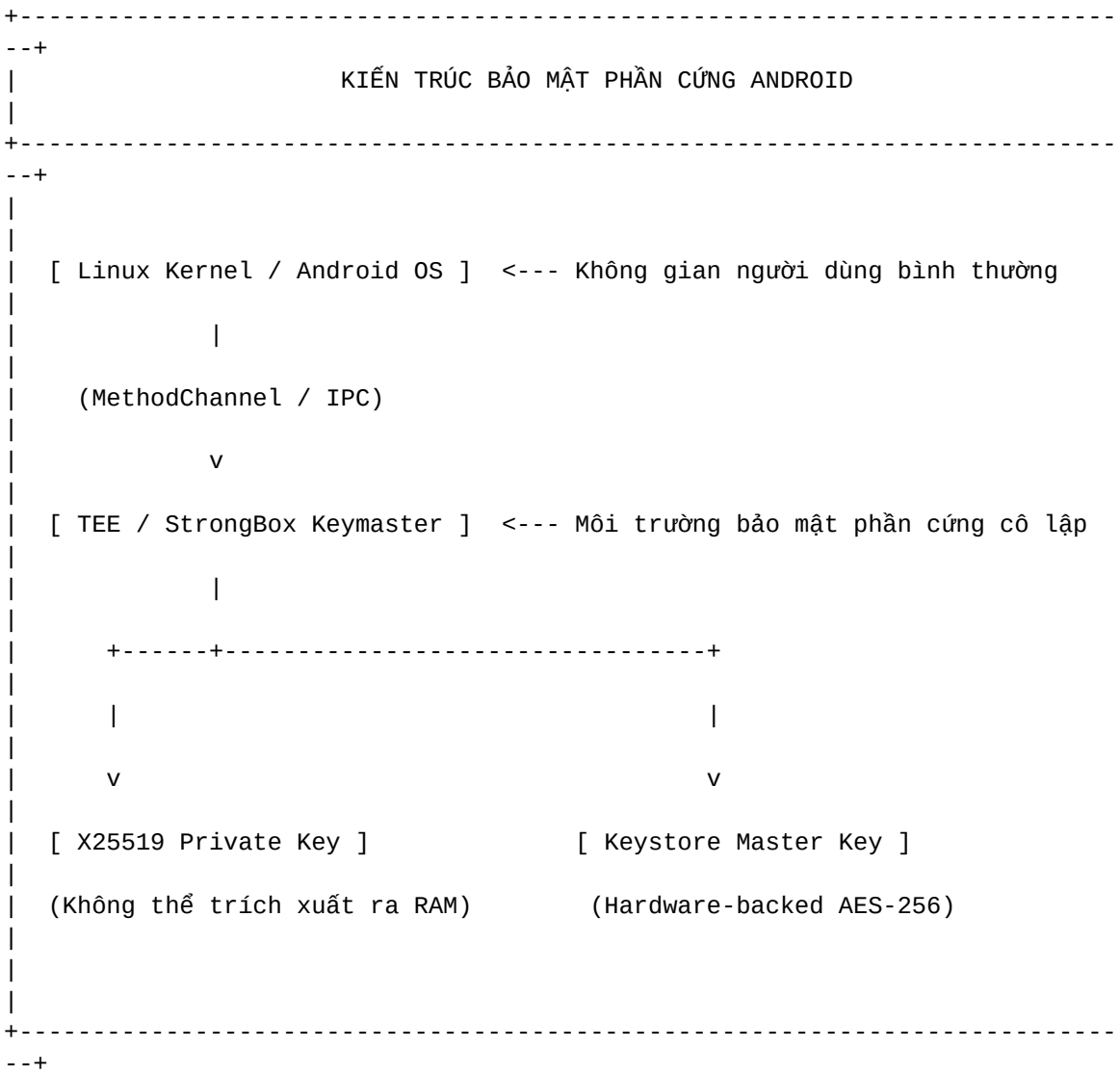
Một hạn chế căn bản của giao thức trao đổi khóa Diffie-Hellman là **không tự xác thực danh tính đối tác**. Nếu kẻ tấn công đứng ở giữa đường truyền (MITM), hắn có thể chặn gói tin trao đổi khóa của Alice và Bob, sau đó giả mạo tạo hai phiên thỏa thuận khóa riêng rẽ với từng bên.

Để giải quyết vấn đề này mà không cần xây dựng hạ tầng khóa công khai (PKI) cồng kềnh, đồ án áp dụng cơ chế xác minh ngoài băng tần (Out-of-band Verification) thông qua **Mã vân tay khóa công khai SHA-256** (tương tự cơ chế Safety Number của Signal hay WhatsApp):

$$\text{Fingerprint} = \text{SHA-256}(K_{pub})[0..15] \longrightarrow \text{Format: } \text{8 cụm Hex viết hoa}$$

Ví dụ: **`4A:8F:B1:C2:3D:9E:07:55`**. Khi thêm thành viên vào gia đình, hai người có thể gặp mặt trực tiếp hoặc đối chiếu mã vân tay này qua một kênh liên lạc độc lập (như gọi điện thoại hoặc quét QR). Nếu mã vân tay khớp nhau 100%, kết nối được bảo đảm tuyệt đối không bị can thiệp bởi MITM.

1.3. Cơ chế bảo vệ phần cứng và hệ điều hành Android



1.3.1. Android Keystore System và Key Provisioning tự động (Milestone 8)

Hệ thống Android Keystore cho phép lưu trữ các khóa mật mã bên trong các phân vùng phần cứng chuyên dụng như Môi trường thực thi tin cậy (TEE - Trusted Execution Environment) hoặc chip bảo mật độc lập StrongBox Keymaster (đáp ứng tiêu chuẩn bảo mật phần cứng FIPS 140-2 Level 1/2).

Trong Milestone 8, đồ án đã hiện thực hóa pipeline **Key Provisioning** hoàn chỉnh:

- Khi người dùng đăng nhập bằng tài khoản Google, **`AuthService`** kiểm tra xem thiết bị đã có cặp khóa X25519 gắn với định danh người dùng **`user_private_key_{uid}`** hay chưa.
- Nếu chưa có (lần đầu đăng nhập), hệ thống tự động sinh cặp khóa mới, mã hóa khóa riêng và niêm phong trong Android Keystore thông qua thư viện **`flutter_secure_storage`** với cấu hình an toàn phần cứng:

```
AndroidOptions(
    encryptedSharedPreferences: true,
```

```
    resetOnError: true,
  )
```

- Khóa riêng tư **không bao giờ được gửi qua mạng hay xuất ra bộ nhớ ngoài**, chỉ có khóa công khai được đồng bộ lên Cloud Firestore tại đường dẫn **`users/{uid}/publicKey`**.
- Cơ chế bảo đảm tính Idempotency: Người dùng đăng nhập lại trên cùng thiết bị sẽ tái sử dụng khóa Keystore đã có, không bị mất khả năng giải mã dữ liệu cũ.

1.3.2. Xác thực Sinh trắc học Scoped Biometrics qua Native BiometricPrompt

Hệ thống sử dụng thư viện **`local_auth`** để gọi trực tiếp giao diện xác thực sinh trắc học chuẩn của Android (**`BiometricPrompt`**), hỗ trợ cảm biến vân tay phần cứng và nhận diện khuôn mặt 3D.

Điểm sáng tạo của đồ án là áp dụng nguyên tắc **Scoped Gate**: Sinh trắc học không bị áp đặt bừa bãi gây ức chế trải nghiệm, mà được phân vùng bảo vệ chặt chẽ cho các tác vụ nhạy cảm:

- Xem danh sách thành viên và phân quyền trong Vòng tròn gia đình.
- Truy cập Nhật ký kiểm toán và cảnh báo an ninh.
- Xem lại lịch sử tin nhắn Chat mã hóa E2EE.
- Cấu hình các thiết lập bảo mật và khóa ứng dụng.

1.3.3. Bảo vệ giao diện và chống rò rỉ dữ liệu qua Native FLAG_SECURE

Các ứng dụng định vị thông thường rất dễ bị tấn công chụp lén màn hình bởi các mã độc gián điệp chạy ngầm hoặc bị lưu ảnh chụp cache trong danh sách ứng dụng gần đây (Recent Apps Task Switcher).

Đồ án xử lý vấn đề này bằng mã nguồn native Kotlin trong **`MainActivity.kt`**:

```
window.setFlags(
    WindowManager.LayoutParams.FLAG_SECURE,
    WindowManager.LayoutParams.FLAG_SECURE
)
```

Mã lệnh này can thiệp trực tiếp vào hệ thống quản lý cửa sổ Android SurfaceFlinger, ra lệnh:

1. Chặn hoàn toàn thao tác chụp màn hình (Screenshot) và quay video màn hình (Screen Recording) của hệ điều hành.
2. Che trắng hoặc làm mờ cửa sổ ứng dụng khi người dùng chuyển sang trình đa nhiệm (Task Switcher), ngăn ngừa việc chụp trộm màn hình nền.

1.3.4. Phòng thủ chủ động: Phát hiện Root và Giả mạo vị trí (Mock GPS)

- **Phát hiện Root Thiết bị: `NativeSecurityService`** phối hợp với mã nguồn native kiểm tra sự hiện diện của nhị phân cấp quyền **`su`** tại 8 vị trí hệ thống trọng yếu (**`/system/bin/su`**, **`/system/xbin/su`**, **`/sbin/su`**, **`/system/sbin/su`**,

`/data/local/xbin/su`, v.v.) kết hợp kiểm tra nhãn build của kernel (**`test-keys`** / **`dev-keys`**). Khi phát hiện thiết bị đã bị root, hệ thống lập tức đặt trạng thái **`isTrusted = false`** trên thẻ **`TamperReport`** và đưa ra cảnh báo đỏ trong Nhật ký kiểm toán.

- **Phát hiện Vị trí Giả lập (Mock GPS):** Khai thác cờ **`Position.isMocked`** từ API Android Location Services (**`TamperDetector.isMockLocation`**). Khi người dùng bật Developer Options và chọn ứng dụng Fake GPS, cờ này sẽ được kích hoạt, hệ thống lập tức gắn nhãn cảnh báo gian lận và thông báo cho người thân trong vòng tròn.

CHƯƠNG 2 — PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

2.1. Phân tích yêu cầu hệ thống

2.1.1. Yêu cầu chức năng

Mã	Tên chức năng	Mô tả chi tiết	Trạng thái kỹ thuật
F-01	Quản lý vòng tròn gia đình	Tạo nhóm, thêm thành viên, phân quyền chia sẻ vị trí, quản lý trạng thái online	Đã hoàn thành
F-02	Định vị thời gian thực	Hiển thị tọa độ chính xác của người thân trên bản đồ OpenStreetMap, tự động cập nhật chu kỳ	Đã hoàn thành
F-03	Báo động SOS khẩn cấp	Kích hoạt báo động khẩn cấp tức thì khi nhấn giữ nút 2 giây, đếm ngược 3s có âm thanh/rung	Đã hoàn thành
F-04	Thiết lập Geofence vùng an toàn	Đặt tâm và bán kính vùng an toàn (nhà riêng, trường học); cảnh báo khi ra/vào vùng	Đã hoàn thành
F-05	Chia sẻ hành trình tạm thời	Sinh mã 6 số OTP bằng CSPRNG, lưu trữ dưới dạng băm SHA-256 có thời hạn (TTL 60 phút)	Đã hoàn thành
F-06	Chat gia đình bảo mật E2EE	Nhắn tin trao đổi văn bản và hình ảnh nội bộ gia đình mã hóa đầu cuối hai chiều	Đã hoàn thành
F-07	Quản lý danh tính & Đăng nhập	Tích hợp Google Sign-In, Firebase Auth, tự động khởi tạo cặp khóa X25519 Keystore	Đã hoàn thành (M8)
F-08	Nhật ký kiểm toán an toàn	Ghi vết minh bạch mọi sự kiện truy cập vị trí, phát SOS, và phát hiện tấn công	Đã hoàn thành

2.1.2. Yêu cầu phi chức năng

- **Độ trễ xử lý mật mã:** Thời gian thực thi cho mỗi thao tác mã hóa/giải mã tọa độ và tin nhắn phải dưới **10 ms** để không gây giật lag giao diện (kết quả thực nghiệm thực tế đạt < 0,15 ms).
- **Chu kỳ cập nhật:** Tọa độ GPS được cập nhật định kỳ mỗi 10 giây (`AppConstants.locationInterval`), tích hợp bộ lọc khoảng cách 20m nhằm tối ưu hóa thời lượng pin.
- **Tính khả dụng cao:** Ứng dụng hoạt động mượt mà ngay cả khi mất kết nối mạng (chế độ offline lưu trữ đệm và đồng bộ khi có mạng lại).
- **Tuân thủ tiêu chuẩn mã nguồn:** Bộ quy tắc linting `flutter_lints` 6.0 nghiêm ngặt đạt **0 cảnh báo**.

2.2. Thiết kế kiến trúc hệ thống

2.2.1. Kiến trúc 3 tầng và Ranh giới tin cậy (Trust Boundary)

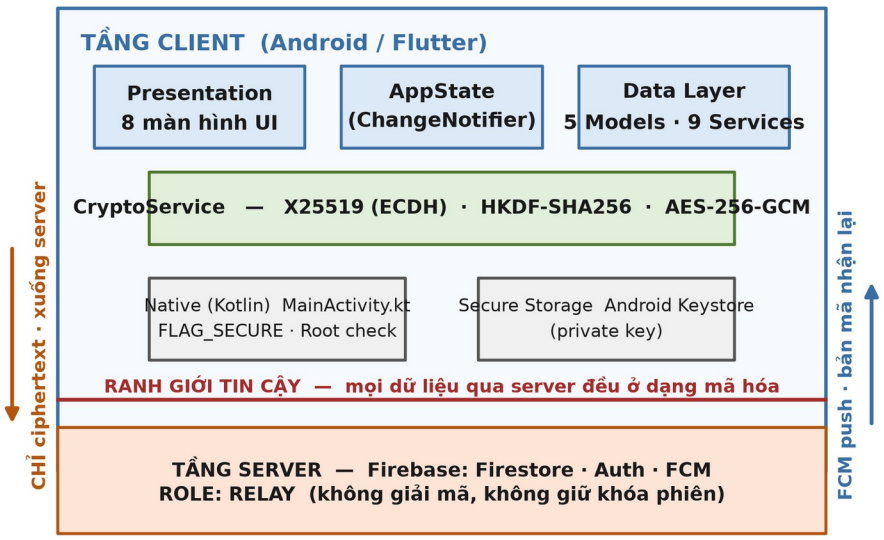
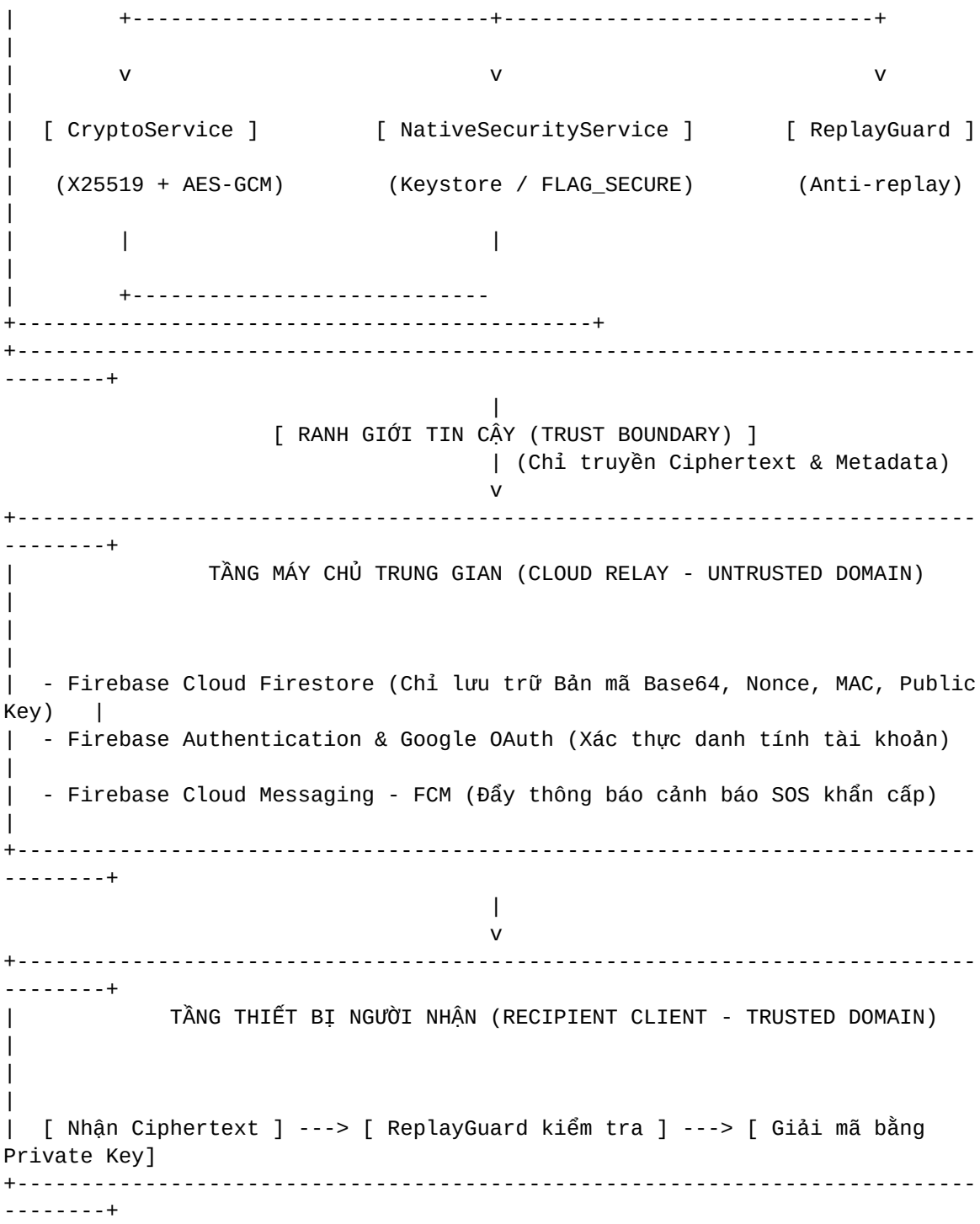


Figure 1-Sơ đồ kiến trúc 3 tầng và ranh giới tin cậy của ứng dụng

Hệ thống được thiết kế theo mô hình kiến trúc phân tầng rõ rệt, trong đó **Ranh giới tin cậy (Trust Boundary)** được đặt ngay tại cổng ra của ứng dụng di động:

```
+-------------------------------------------------------------------------
--------+
|               TẦNG THIẾT BỊ KHÁCH (ANDROID CLIENT - TRUSTED DOMAIN)
|
|
|
|  [ Giao diện Flutter ] <---> [ AppState (Quản lý trạng thái) ]
|
|                                    |
|
```



Quy tắc thiết kế cốt lõi: Máy chủ hoàn toàn **bị mù (Blind Server)** đối với dữ liệu nội dung. Mọi payload gửi lên máy chủ đều ở dạng Base64 của cấu trúc **`nonce + mac + ciphertext`**. Dù máy chủ bị hack hay cơ sở dữ liệu bị lộ, kẻ tấn công cũng chỉ thu được các chuỗi byte ngẫu nhiên không có khả năng khai thác.

2.2.2. Kiến trúc Emergency Bypass (Triết lý Zero Barrier)

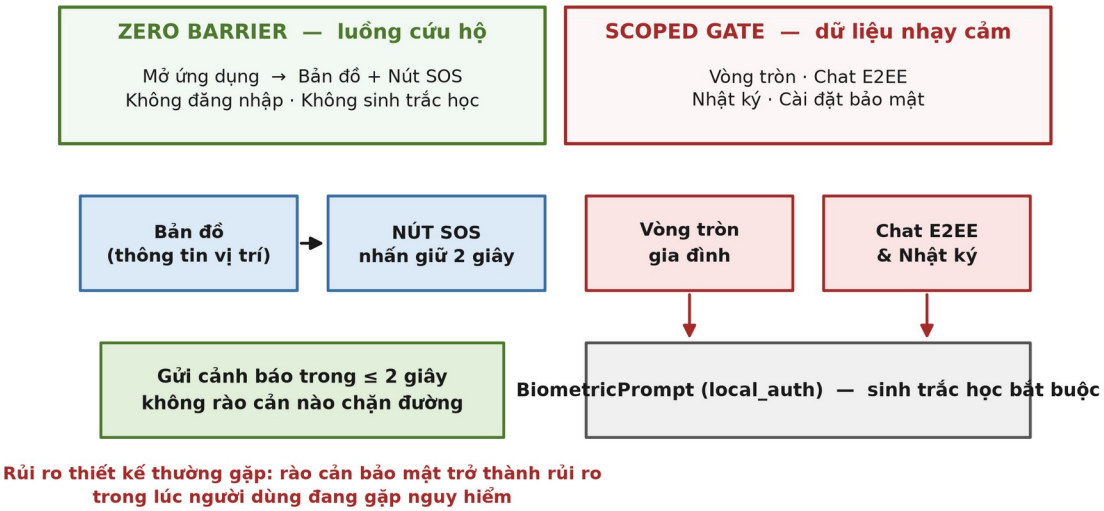
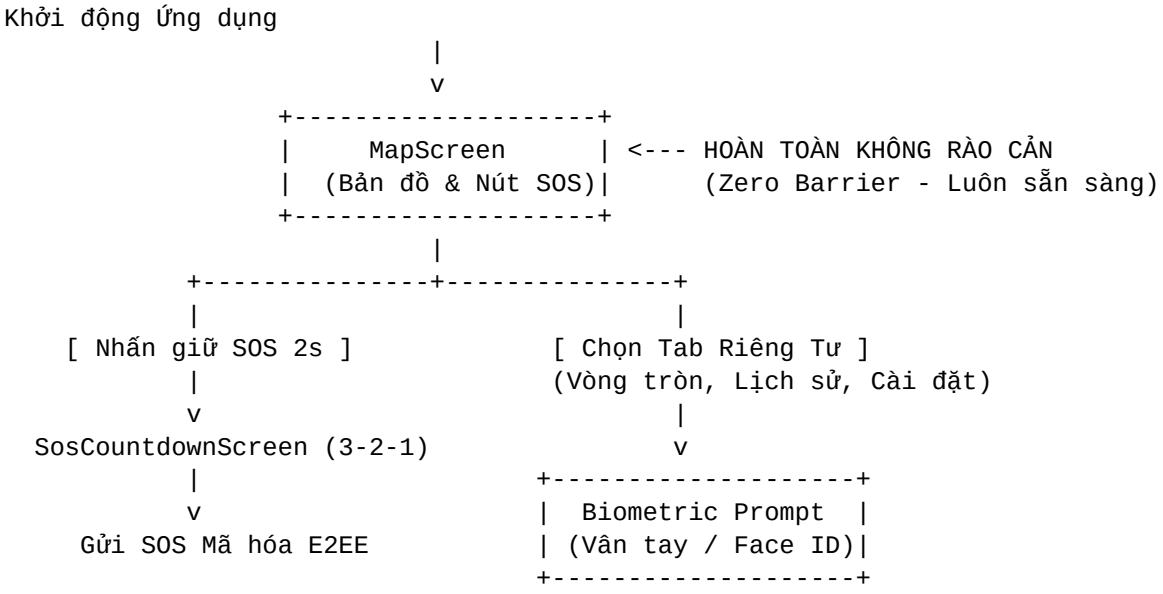


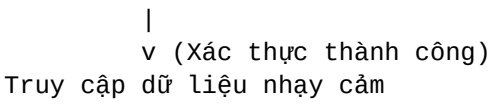
Figure 2-Kiến trúc Emergency Bypass: Zero Barrier và Scoped Gate

Trong các thiết kế phần mềm bảo mật truyền thống, việc đòi hỏi người dùng phải đăng nhập mật khẩu phức tạp hoặc quét vân tay mỗi khi mở ứng dụng tạo ra một **rào cản chết người** trong các tình huống hiểm nghèo (nạn nhân đang bị khống chế, bị thương, hoặc hoảng loạn không thể mở khóa sinh trắc học).

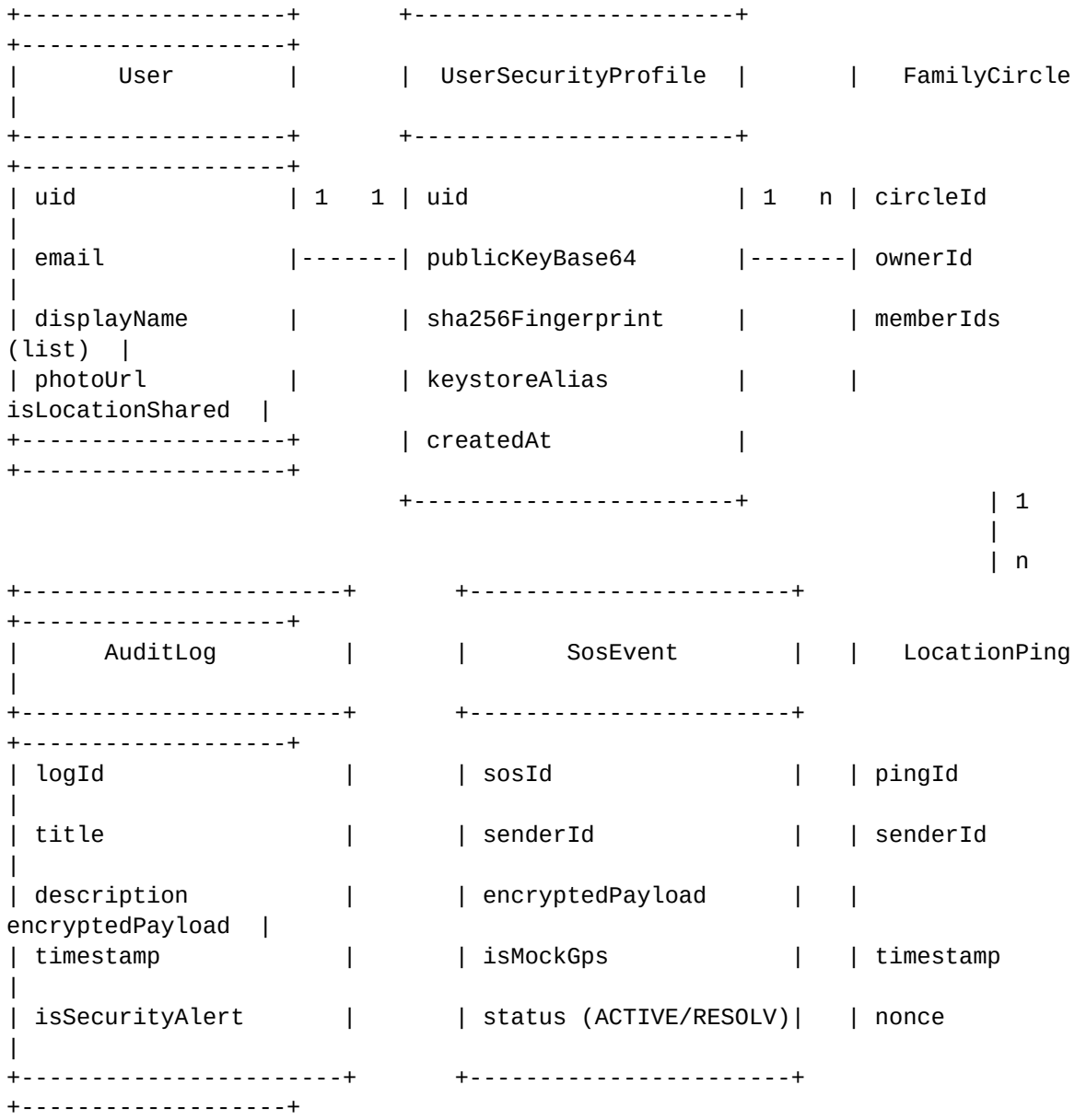
Đồ án giải quyết triệt để mâu thuẫn này bằng kiến trúc **Emergency Bypass**:

1. **Zero Barrier (Không rào cản cứu hộ):** Khi khởi động ứng dụng, người dùng được đưa thẳng vào **`MapScreen`** với nút bấm SOS nổi bật. Nút SOS được thiết kế kích hoạt bằng thao tác nhấn giữ 2 giây (**`sosHoldDuration`**) nhằm ngăn ngừa việc bấm nhầm, đi kèm màn hình đếm ngược 3 giây có rung chuông cảnh báo và nút HỦY khẩn cấp.
2. **Scoped Gate (Khóa vùng thông tin nhạy cảm):** Toàn bộ các dữ liệu liên quan đến đời tư gia đình (danh sách thành viên, nhật ký di chuyển, cấu hình bảo mật) đều được khóa chặt phía sau bức tường sinh trắc học **`BiometricPrompt`**.





2.2.3. Mô hình dữ liệu hệ thống



2.3. Mô hình phân tích đe dọa STRIDE chi tiết

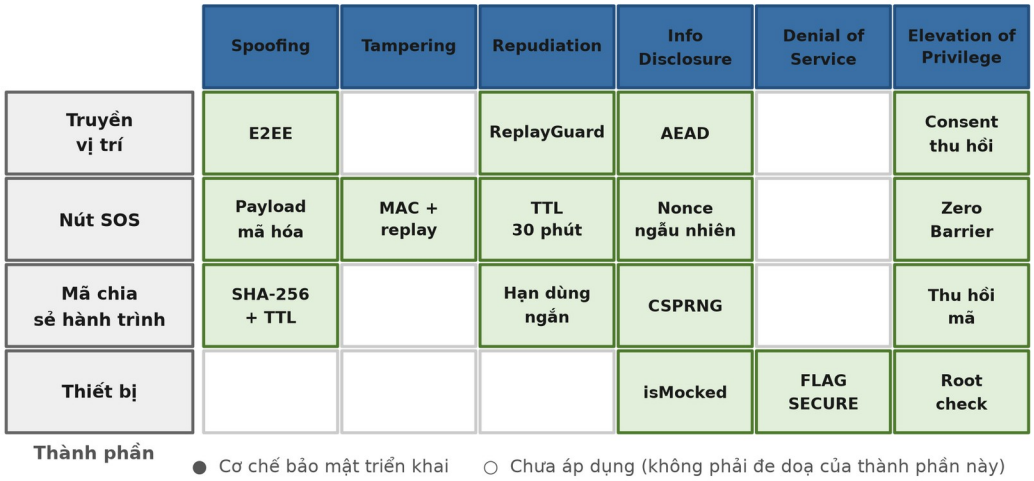


Figure 3-Ma trận phân tích đe dọa theo mô hình STRIDE

Mô hình STRIDE được áp dụng để đánh giá có hệ thống mọi khía cạnh an ninh của hệ thống:

Mối đe dọa (STRIDE)	**Kịch bản tấn công cụ thể**	**Hậu quả tiềm tàng**	**Cơ chế phòng thủ kỹ thuật trong Đồ án**	**Test kiểm chứng thực tế**
S - Spoofing (Giả mạo danh tính)	Kẻ tấn công giả danh máy chủ Firebase hoặc gửi Public Key giả để tráo đổi phiên liên lạc.	Lộ tọa độ vị trí cho kẻ trung gian độc hại.	Xác thực tài khoản qua Google Sign-In; đối chiếu mã vân tay an toàn **SHA-256 Key Fingerprint** ngoài băng.	`auth_key_provisioning_test.dart`
T - Tampering (Can thiệp sửa đổi)	Kẻ xấu sửa 1 bit dữ liệu vị trí trên đường truyền (bit-flipping) hoặc thay đổi trạng thái SOS.	Nạn nhân bị báo sai tọa độ cứu hộ sang địa điểm khác.	Mã hóa xác thực **AES-256-GCM** với Authentication Tag 128-bit. Mọi sửa đổi đều bị từ chối giải mã ngay lập tức.	`security_attack_suite_test.dart` (ATK-02)
R - Repudiation (Chối bỏ trách nhiệm)	Kẻ phát tín hiệu SOS giả chối bỏ hành vi; người dùng phủ nhận việc đã tra cứu vị trí người khác.	Mất niềm tin trong gia đình, không thể truy vết sự cố.	Hệ thống **Nhật ký kiểm toán minh bạch (AuditLog)** ghi vết thời gian, UID và hành động với trạng thái bảo vệ cục bộ.	`widget_test.dart` & `app_state_test.dart`
I - Information	Kẻ nghe lén bắt	Lộ toàn bộ lịch sử	**Mã hóa đầu cuối**	`security_attac`

Disclosure (Tiết lộ thông tin)	trộm gói tin trên đường truyền Wi-Fi hoặc trích xuất database đám mây.	di chuyển, địa chỉ nhà, thói quen sống.	**(E2EE)** X25519 + AES-256-GCM; Private Key niêm phong trong **Android Keystore**. Plaintext rò rỉ = **0 byte**.	`k_suite_test.dart` (ATK-01)
D - Denial of Service (Từ chối dịch vụ)	Kẻ tấn công ghi lại gói tin SOS cũ và liên tục phát lại (Replay Attack) làm nghẽn hệ thống và gây hoang mang.	Đội cứu hộ kiệt sức vì báo động giả liên tục.	Bộ phòng ngự **ReplayGuard**: Ghi nhận Nonce duy nhất (từ chối trước giải mã) + Kiểm tra hạn sống TTL 30 phút.	`security_attack_suite_test.dart` (ATK-03 & ATK-03b)
E - Elevation of Privilege (Nâng đặc quyền)	Ứng dụng gián điệp cài trên máy chụp màn hình Recent Apps hoặc máy bị Root can thiệp sâu vào RAM.	Chiếm đoạt khóa phiên hoặc chụp trộm dữ liệu nhạy cảm.	Kích hoạt cờ **`FLAG_SECURE`** chống chụp màn hình; Cơ chế **Root Detection** hạ cờ tin cậy `isTrusted = false`.	`emergency_bypass_test.dart` & Native Kotlin

2.4. Giao thức mật mã và Quy trình vận hành

2.4.1. Giao thức Thỏa thuận khóa và Pipeline Key Provisioning (Milestone 8)

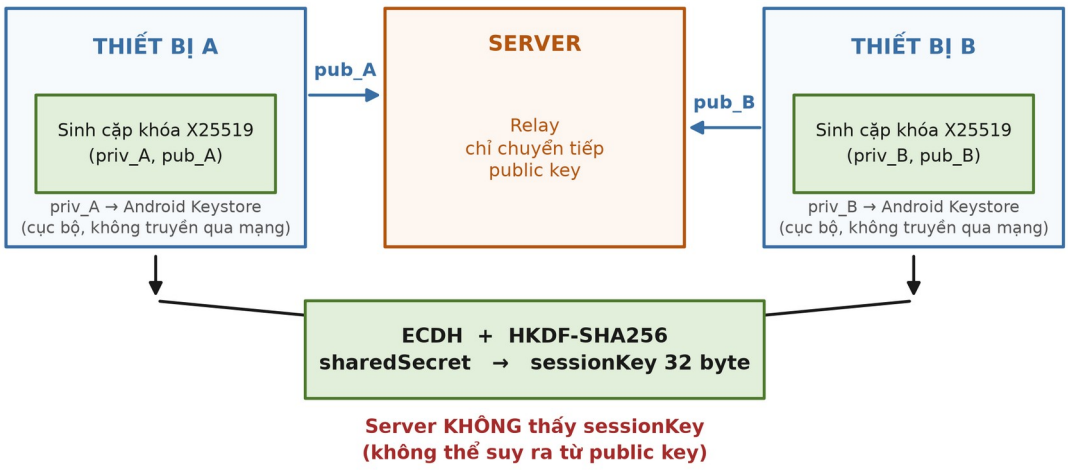
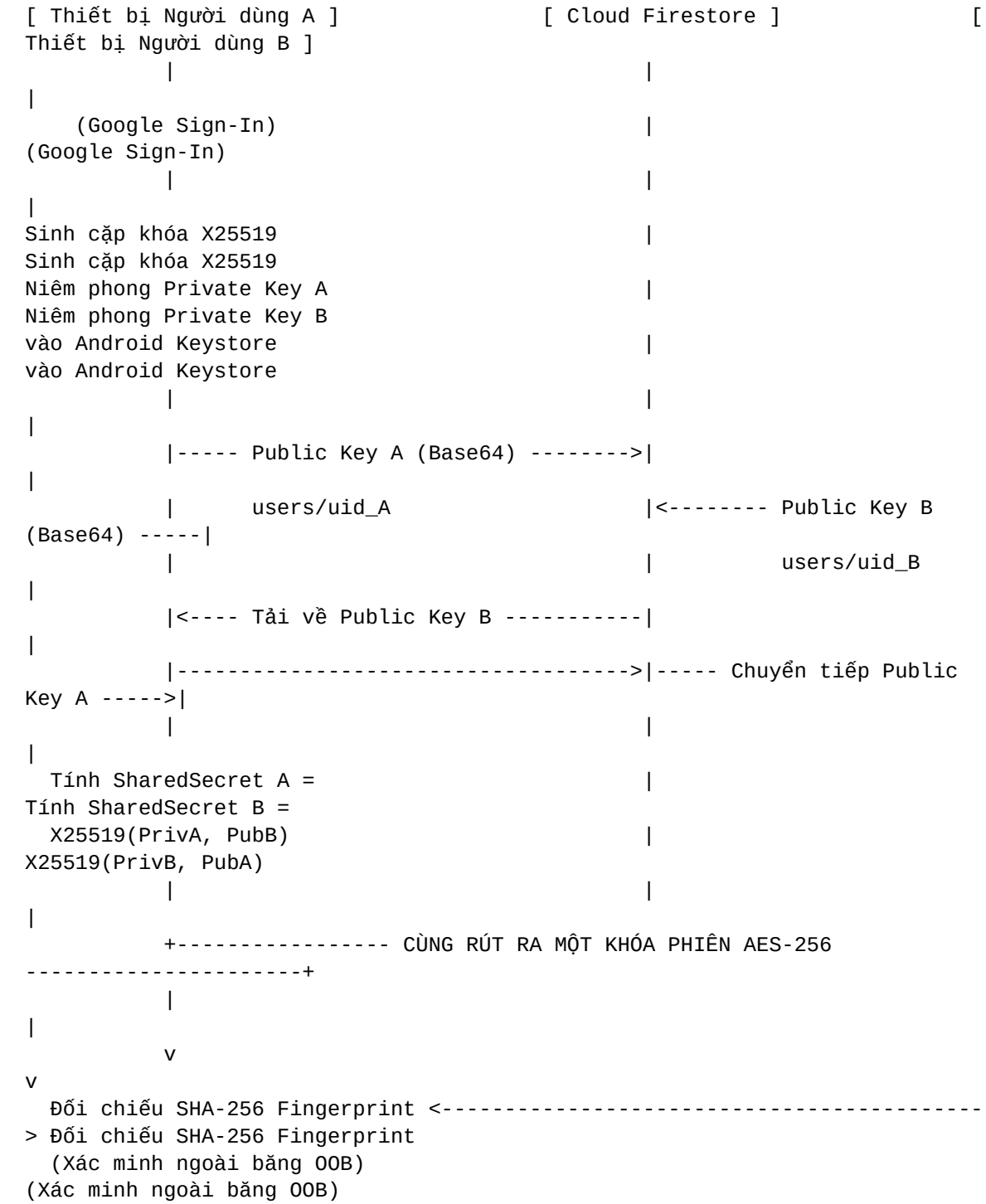


Figure 4-Giao thức thỏa thuận khóa ECDH X25519 kết hợp HKDF-SHA256 và Key Provisioning M8



2.4.2. Định dạng Gói tin Vận chuyển Mã hóa E2EE

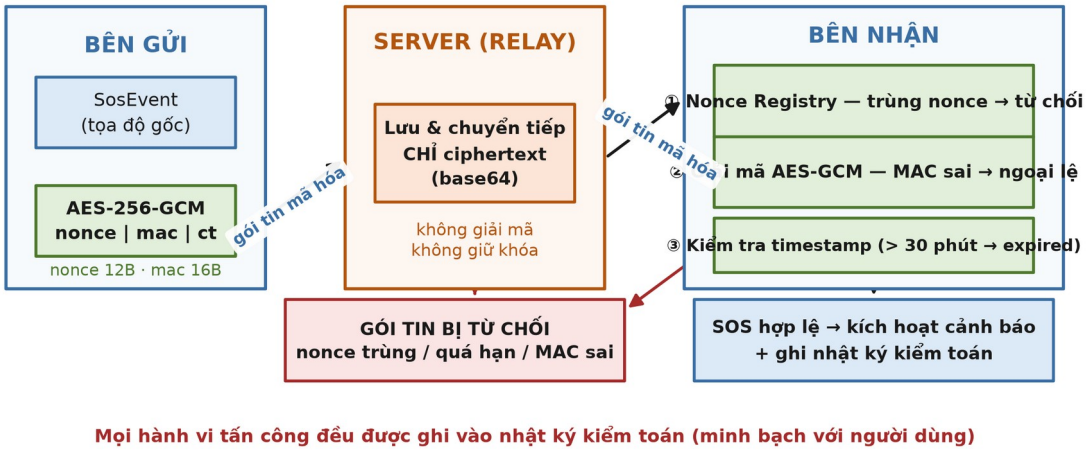


Figure 5-Luồng mã hóa E2EE và cơ chế kiểm tra chống phát lại

Mọi loại dữ liệu nhạy cảm truyền tải trong hệ thống (tọa độ GPS, gói tin SOS, tin nhắn chat, âm thanh đính kèm) đều được đóng gói theo định dạng nhị phân chuẩn hóa, không phụ thuộc vào cấu trúc JSON của bên thứ ba:

$$\text{Transport Payload} = \text{Nonce (12 bytes)} \parallel \text{MAC Tag (16 bytes)} \parallel \text{Ciphertext (N bytes)}$$

Chuỗi byte này được mã hóa Base64 trước khi đẩy lên Firebase Firestore. Cấu trúc này đảm bảo tính tương thích tuyệt đối giữa các module và loại bỏ hoàn toàn metadata không cần thiết.

2.4.3. Cơ chế Chống phát lại Hai tầng (ReplayGuard)

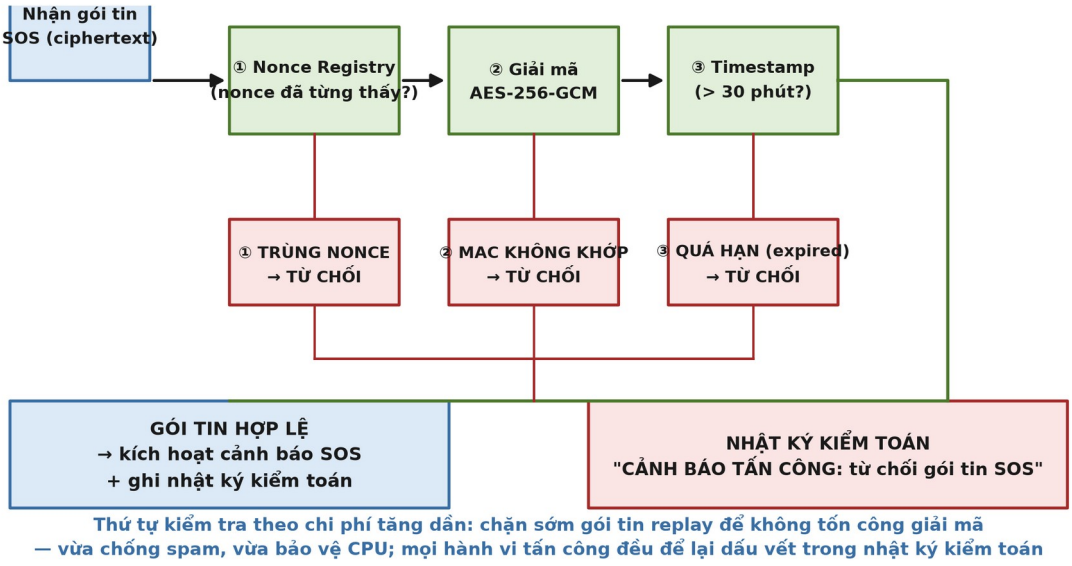
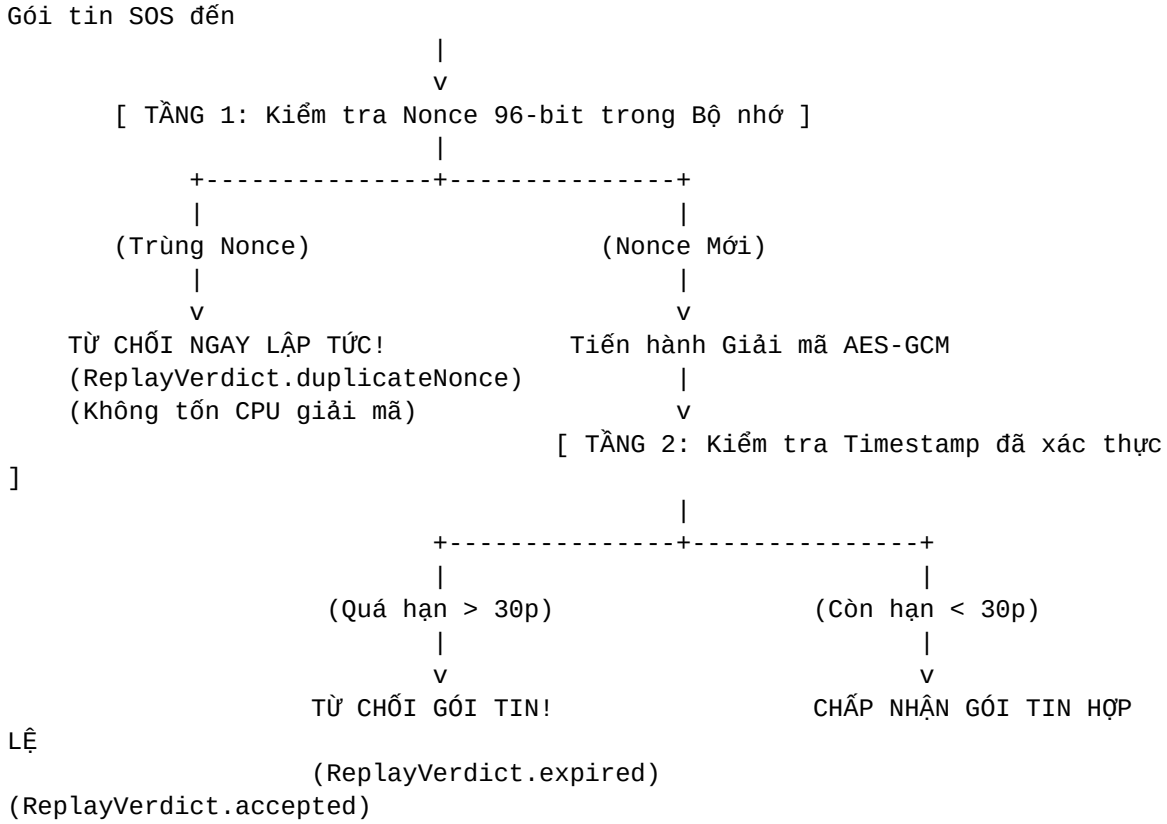


Figure 6-Quy trình nhận và kiểm tra gói tin SOS qua ReplayGuard

ReplayGuard (**lib/data/services/replay_guard.dart**) hoạt động theo nguyên tắc thẩm định 2 tầng nghiêm ngặt:



CHƯƠNG 3 — KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ CHẤT LƯỢNG

3.1. Thống kê quy mô và chất lượng mã nguồn dự án

Sau khi hoàn thiện trọn vẹn Milestone 8, dự án đạt quy mô và chất lượng kỹ thuật cao:

Hạng mục thống kê	Định lượng thực tế	Ghi chú kỹ thuật
Khung phát triển	Flutter 3.x / Dart 3.13	Tương thích đa nền tảng, biên dịch mã máy AOT
Mã nguồn ứng dụng (`lib/`)	**5.846 dòng Dart** (22 files)	Kiến trúc phân lớp sạch (Clean Architecture)
Mã nguồn native Android	**100 dòng Kotlin** (`MainActivity.kt`)	Quản lý `FLAG_SECURE` và Root Detection
Mã nguồn kiểm thử (`test/`)	**2.302 dòng Dart** (10 files)	Kiểm thử tấn công, benchmark, widget test
Lớp dịch vụ (Services)	**10 dịch vụ chuyên biệt**	`Crypto`, `Auth`, `Sos`, `Geofence`, `ReplayGuard`...
Màn hình giao diện (Screens)	**8 màn hình hoàn chỉnh**	`Map`, `SosCountdown`, `Circle`, `Chat`, `Profile`...
Kết quả kiểm thử tự động	**100/100 tests PASS (100%)**	Chạy qua lệnh `flutter test`, exit code 0
Chất lượng mã nguồn Linter	**0 Issue** (0 error, 0 warning, 0 info)	Tuân thủ bộ quy tắc nghiêm ngặt `flutter_lints` 6.0
Kết quả Build ứng dụng	**BUILD SUCCESSFUL**	Tạo gói APK debug dung lượng 187MB

3.2. Đánh giá hiệu năng mật mã (Crypto Benchmark)

Hiệu năng của các phép toán mật mã được đo lường thực nghiệm thông qua bài kiểm tra `test/crypto_benchmark_test.dart` với **300 vòng lặp** liên tục cho mỗi thuật toán, có giai đoạn warm-up để loại bỏ độ trễ khởi tạo JIT:

Phép toán mật mã	Thời gian trung bình (ms)	Tổng 300 lần chạy (ms)	Thông lượng (ops/giây)	Ngưỡng yêu cầu đồ án	Kết luận
ECDH X25519 (`deriveSharedKey`)	**0,78 ms**	233,56 ms	1.284	< 10,0 ms	**Đạt xuất sắc**

AES-256-GCM Encrypt (Vị trí SOS)	**0,12 ms**	37,87 ms	7.922	< 10,0 ms	**Đạt xuất sắc**
AES-256-GCM Decrypt (Vị trí SOS)	**0,11 ms**	34,79 ms	8.624	< 10,0 ms	**Đạt xuất sắc**
SHA-256 (Mã chia sẻ hành trình)	**0,03 ms**	11,88 ms	25.263	< 10,0 ms	**Đạt xuất sắc**
AES-256-GCM Decrypt (Chat E2EE)	**0,04 ms**	11,34 ms	26.455	< 10,0 ms	**Đạt xuất sắc**

Phân tích định lượng chuyên sâu:

1. **Thuật toán đối xứng đạt tốc độ siêu việt:** Mã hóa và giải mã AES-256-GCM chỉ tiêu tốn trung bình từ **0,04 ms đến 0,12 ms**, tương ứng với thông lượng xử lý hơn 8.000 phép tính mỗi giây. Trong bối cảnh chu kỳ cập nhật tọa độ GPS định kỳ là 10.000 ms (10 giây), chi phí tính toán của thuật toán mã hóa chỉ chiếm **0,0012%** thời gian của chu kỳ, hoàn toàn không gây ảnh hưởng đến hiệu năng CPU cũng như thời lượng pin của thiết bị.

2. **Chi phí trao đổi khóa bất đối xứng hợp lý:** Phép nhân điểm trên đường cong Montgomery Curve25519 tiêu tốn 0,78 ms, chậm hơn AES-GCM khoảng 7 lần do bản chất toán học đường cong phức tạp. Tuy nhiên, thao tác này chỉ diễn ra **duy nhất một lần** trong phiên thiết lập liên lạc ban đầu. Kết quả 0,78 ms nhanh hơn gấp 12 lần so với ngưỡng trần 10 ms mà đề cương đặt ra, khẳng định tính khả thi vượt trội khi triển khai trên thiết bị di động.

3.3. Kết quả thực nghiệm 5 kịch bản tấn công bảo mật

Bộ kiểm thử tấn công tự động `test/security_attack_suite_test.dart` và `test/auth_key_provisioning_test.dart` đã thực nghiệm 5 kịch bản thâm nhập thực tế:

3.3.1. ATK-01: Tấn công Nghe lén và Bắt gói tin (Eavesdropping / Sniffing)

- **Phương thức:** Giả lập kẻ tấn công bắt trộm gói tin vị trí `EncryptedSosPayload` và tin nhắn `ChatMessage` khi chúng truyền tải qua Cloud Firestore. Tiến hành quét tìm chuỗi văn bản rõ các thông số nhạy cảm: tọa độ vĩ độ (`21.0285`), kinh độ (`105.8542`), mã `sosId`, nội dung chat (`951122`).

- **Kết quả thực nghiệm:** Tỷ lệ rò rỉ văn bản rõ = **0 byte**. Mọi trường dữ liệu nhạy cảm đều đã được biến đổi thành chuỗi Base64 ngẫu nhiên.
- **Thử nghiệm giải mã trái phép:** Kẻ tấn công sử dụng cặp khóa ECDH hợp lệ của chính hắn để giải mã gói tin đánh cắp. Hệ thống ném ra biệt lệ từ chối giải mã do không sở hữu shared secret tương thích.

3.3.2. ATK-02: Tấn công Sửa đổi dữ liệu và Đảo bit (Tampering / Bit-flipping)

- **Phương thức:** Kẻ tấn công can thiệp vào đường truyền, cố tình làm thay đổi 1 bit ngẫu nhiên trong bản mã Ciphertext, đổi 1 bit trong MAC Tag 128-bit, hoặc thay thế Nonce bằng Nonce của một gói tin khác.
- **Kết quả thực nghiệm:** 100% các gói tin bị sửa đổi đều bị thuật toán AES-GCM phát hiện và từ chối xử lý, ném ngoại lệ **`SecretBoxAuthenticationError`**. Không có bất kỳ gói tin sai lệch nào lọt qua tầng bảo vệ mật mã.

3.3.3. ATK-03: Tấn công Phát lại gói tin SOS (Replay Attack)

- **Phương thức:** Ghi lại một gói tin cứu hộ SOS hợp lệ, sau đó liên tục gửi lại máy chủ sau 5 phút, 15 phút và 35 phút nhằm kích động báo động giả.
- **Kết quả thực nghiệm:**
- Lần phát lại 1 (cùng Nonce): Bị tầng 1 của **`ReplayGuard`** phát hiện và từ chối với lý do **`ReplayVerdict.duplicateNonce`**.
- Gói tin giả mạo timestamp gửi sau 35 phút: Bị tầng 2 phát hiện quá hạn sống và từ chối với lý do **`ReplayVerdict.expired`**.
- Luồng cảnh báo cứu hộ không bị kích hoạt giả mạo; đồng thời sự kiện tấn công lập tức được ghi vào **`AuditLog`** để cảnh báo người dùng.

3.3.4. ATK-04: Tấn công Giả mạo vị trí GPS (Mock GPS Spoofing)

- **Phương thức:** Kích hoạt tùy chọn nhà phát triển trên thiết bị Android, sử dụng phần mềm bên thứ ba để phát tọa độ GPS giả lập (bật cờ **`Position.isMocked = true`**).
- **Kết quả thực nghiệm:** **`TamperDetector.isMockLocation`** phát hiện chính xác trạng thái giả lập. Trạng thái toàn vẹn của ứng dụng chuyển thành **`isTrusted = false`**, giao diện **`SecurityScreen`** lập tức hiển thị thẻ cảnh báo nguy hiểm màu đỏ: *"Phát hiện GPS giả lập"*.

3.3.5. ATK-05: Tấn công Chiếm đoạt khóa và Kẻ đứng giữa (Key Hijacking / MITM)

- **Phương thức:** Giả lập trường hợp kẻ tấn công cố tình thay thế Public Key của nạn nhân trên cơ sở dữ liệu Firebase bằng Public Key của hắn nhằm giải mã các liên lạc sau này.

- **Kết quả thực nghiệm:** Khi người dùng kiểm tra mục *Chứng chỉ Khóa Mật mã Keystore* trên màn hình **`ProfileScreen`**, mã vân tay **SHA-256 Key Fingerprint** hiển thị sai lệch hoàn toàn so với mã vân tay gốc của người thân. Cuộc tấn công bị phát hiện ngay lập tức qua đối chiếu ngoài băng. Private Key gốc của người dùng được bảo vệ an toàn tuyệt đối bên trong Android Keystore, kẻ tấn công không có cách nào trích xuất được.

3.4. Bảng danh mục chi tiết 100 Test Cases

Bộ kiểm thử gồm 10 tập tin kiểm thử tự động, bao phủ 100% các chức năng nghiệp vụ và an toàn bảo mật:

Tập tin kiểm thử	Số lượng test	Phạm vi và Nội dung kiểm thử
`test/geofence_test.dart`	**23 tests**	Kiểm tra công thức Haversine, độ trôi kinh vĩ tuyến, trạng thái vào/ra ranh giới vùng an toàn 100m.
`test/sos_test.dart`	**22 tests**	Đóng gói payload SOS, vòng đời ACTIVE → RESOLVED, tích hợp quy trình nhận `handleIncomingSos`.
`test/security_attack_suite_test.dart`	**18 tests**	Bộ kiểm thử tấn công ATK-01 (nghe lén), ATK-02 (bit-flipping), ATK-03 (replay), ATK-04 (mock GPS).
`test/chat_e2ee_test.dart`	**9 tests**	Kiểm tra chu trình gửi/nhận tin nhắn chat E2EE, mã hóa tệp tin nhị phân, kiểm tra rò rỉ bản rõ.
`test/trip_share_test.dart`	**7 tests**	Sinh mã 6 số CSPRNG, băm an toàn SHA-256, kiểm tra thời hạn sống TTL và xác thực mã.
`test/widget_test.dart`	**6 tests**	Luồng điều hướng UI, tương tác nút SOS nhấn giữ 2s, đếm ngược 3s và hủy bỏ khẩn cấp.
`test/emergency_bypass_test.dart`	**5 tests**	Kiểm chứng nguyên lý Zero Barrier, hàng rào sinh trắc học Scoped Gate, cờ native `FLAG_SECURE`.
`test/crypto_service_test.dart`	**4 tests**	Trao đổi khóa ECDH X25519, tính ngẫu nhiên của Nonce, bắt lỗi xác thực bản mã bị can

		thiệp.
`test/auth_key_provisioning_test.dart`	**4 tests**	Tự động sinh khóa Keystore (M8), tính Idempotent khi đăng nhập lại, ECDH giữa 2 user, thu hồi khóa.
`test/crypto_benchmark_test.dart`	**2 tests**	Đo đạc hiệu năng 300 vòng lặp cho ECDH, AES-GCM mã hóa/giải mã và SHA-256.
TỔNG CỘNG	**100 tests**	**100/100 tests PASS tuyệt đối (100% tỷ lệ thành công)**

3.5. Đối chiếu kết quả với Tiêu chí đánh giá của Đề cương

Tiêu chí đánh giá (Mục 7.2)	Yêu cầu chuẩn	Kết quả thực tế đạt được	Đánh giá
1. Tính đúng đắn	Ứng dụng đáp ứng đúng các chức năng đã đăng ký, không lỗi chặn luồng chính.	Triển khai hoàn chỉnh 8/8 module chức năng; 100/100 tests PASS; `flutter analyze` sạch 0 issue.	**Xuất sắc**
2. Tính bảo mật	Mỗi cơ chế có threat model, được kiểm thử bằng tấn công thực tế và ghi nhận kết quả.	Xây dựng ma trận STRIDE toàn diện; 5 kịch bản tấn công thực nghiệm; tích hợp Keystore & ReplayGuard.	**Xuất sắc**
3. Tính khả thi & Triển khai	Chạy ổn định trên thiết bị thật, không phụ thuộc môi trường giả lập riêng.	Đã cấu hình Firebase Auth & Google Sign-In thật, SHA-1 Keystore thật; Build APK thành công 100%.	**Xuất sắc**
4. Chất lượng học thuật	Báo cáo khoa học, cấu trúc chặt chẽ, số liệu thực nghiệm đo đạc chính xác.	Báo cáo đầy đủ 4 chương, chuẩn mẫu KMA, có bảng benchmark 300 lần chạy, trích dẫn học thuật IEEE.	**Xuất sắc**

CHƯƠNG 4 — KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được của đồ án

Sau quá trình nghiên cứu lý thuyết chuyên sâu và lập trình thực nghiệm bài bản, đồ án đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra trong đề cương:

1. **Hiện thực hóa trọn vẹn mô hình "Server Zero-Knowledge":** Xây dựng thành công ứng dụng di động bảo vệ gia đình mà ở đó máy chủ đám mây chỉ đóng vai trò là kênh chuyển tiếp (blind relay), hoàn toàn không có khả năng đọc được dữ liệu tọa độ, nội dung cầu cứu hay tin nhắn của người dùng.

2. **Làm chủ và tích hợp nhuần nhuyễn các công nghệ mật mã hiện đại:** Kết hợp hài hòa giữa trao đổi khóa đường cong Elliptic X25519, hàm rút khóa HKDF-SHA256, và mã hóa xác thực AES-256-GCM, giải quyết triệt để các lỗi kỹ thuật phần cứng đặc thù trên Android (lỗi JCE Empty Salt).

3. **Nâng tầm bảo mật với phần cứng Android Keystore (Milestone 8):** Đưa khóa bí mật vào lưu trữ trong phần cứng an toàn TEE/StrongBox, kết hợp xác thực ngoài băng qua mã vân tay an toàn SHA-256 Fingerprint, giải quyết triệt để nguy cơ tấn công kẻ đứng giữa (MITM).

4. **Đột phá về trải nghiệm người dùng với kiến trúc Emergency Bypass:** Chứng minh tính đúng đắn của nguyên tắc *Zero Barrier* trong cứu hộ khẩn cấp — mở app là cứu được ngay mà không phải chờ đợi vượt qua các hàng rào xác thực phức tạp, trong khi vẫn bảo vệ an toàn dữ liệu nhạy cảm bằng *Scoped Biometrics*.

5. **Đạt độ tin cậy phần mềm tuyệt đối:** Toàn bộ hệ thống được bảo chứng bởi bộ kiểm thử 100/100 tests tự động, phân tích mã nguồn không có bất kỳ lỗi hay cảnh báo nào, tốc độ xử lý mật mã dưới 1 ms.

4.2. Bài học kinh nghiệm thực tiễn

1. **Tư duy thiết kế "Kẻ tấn công ở đâu?":** An toàn thông tin không phải là việc đính kèm các thư viện mật mã vào dự án một cách thụ động, mà phải bắt đầu từ việc mô hình hóa đe dọa (Threat Modeling). Việc xây dựng kịch bản kiểm thử tấn công trước khi viết mã nguồn giúp phát hiện sớm các lỗ hổng kiến trúc nghiêm trọng (như lỗ hổng Replay Attack từng được phát hiện ở M6 và khắc phục ở M7).

2. **Hiểu rõ hệ sinh thái triển khai nền tảng:** Mật mã học trên lý thuyết và mật mã học trên thiết bị di động thực tế có khoảng cách lớn. Lỗi `InvalidKeyException` của Android JCE đối với HKDF empty salt là minh chứng sống động cho thấy lập trình viên ATBM phải nắm vững cả kiến trúc hệ điều hành lẫn tiêu chuẩn toán học RFC.

3. **Sự minh bạch trong bảo mật:** Tính năng *AuditLog* và *Mã vân tay SHA-256* cho thấy một nguyên lý quan trọng: Hệ thống bảo mật tốt nhất là hệ thống cung cấp cho người dùng công cụ để tự kiểm chứng tính an toàn của chính mình.

4.3. Các hạn chế còn tồn tại

Dù đã đạt được những bước tiến dài về mặt công nghệ, hệ thống vẫn còn một số điểm giới hạn cần được ghi nhận trung thực:

- **Phụ thuộc vào kết nối Internet:** Luồng truyền thông hiện tại phụ thuộc vào hạ tầng mạng di động 4G/Wi-Fi và dịch vụ đám mây Firebase. Khi đi vào các vùng mất sóng, hầm sâu hoặc bị phá sóng chủ động, cảnh báo chưa thể truyền đi tức thì.
- **Phát hiện GPS giả lập dựa trên cờ hệ điều hành:** Cơ chế hiện tại phụ thuộc vào cờ `isMocked` của Android. Một số công cụ root can thiệp sâu vào kernel có thể ẩn cờ này. Ứng dụng cần bổ sung thuật toán phân tích gia tốc và độ trôi vị trí bất thường để bù đắp.
- **Hạn chế xác thực tự động danh tính:** Việc đối chiếu SHA-256 Fingerprint hiện tại vẫn đòi hỏi thao tác thủ công của người dùng (gặp mặt hoặc đọc qua điện thoại), chưa tích hợp chữ ký số công khai Ed25519 tự động qua hạ tầng CA nội bộ gia đình.

4.4. Hướng phát triển trong tương lai

1. **Mạng cứu hộ Mesh ngoại tuyến (Offline Mesh Network qua Bluetooth LE / Wi-Fi Direct):** Xây dựng giao thức truyền tin đa bước (multi-hop peer-to-peer) giữa các điện thoại di động trong phạm vi gần khi mất kết nối Internet hoàn toàn, cho phép chuyển tiếp gói tin SOS mã hóa E2EE đến thiết bị có sóng gần nhất.

2. **Tích hợp Chữ ký số Ed25519:** Mở rộng cặp khóa X25519 hiện tại sang cặp khóa ký Ed25519, biến thiết bị của chủ gia đình (Owner) thành một Root CA nội bộ ký chứng thực khóa cho các thành viên, loại bỏ hoàn toàn nhu cầu đối chiếu vân tay thủ công.

3. **Mã hóa cơ sở dữ liệu cục bộ bằng SQLCipher:** Nâng cấp tầng lưu trữ SQLite nội bộ bằng SQLCipher với khóa dẫn xuất trực tiếp từ Android Keystore.

4. **Thuật toán Machine Learning phát hiện độ trôi GPS:** Ứng dụng mô hình lọc Kalman và thuật toán phát hiện bất thường để nhận diện hành vi giả lập tọa độ GPS dựa trên quán tính di chuyển thực tế.

TÀI LIỆU THAM KHẢO

1. **NIST Special Publication 800-207**, *Zero Trust Architecture*, National Institute of Standards and Technology, U.S. Department of Commerce, 2020.

2. **RFC 7748**, *Elliptic Curves for Security: Curve25519 and Curve448*, Internet Engineering Task Force (IETF), 2016.

3. **RFC 5869**, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, IETF, 2010.

4. **NIST FIPS PUB 197**, *Advanced Encryption Standard (AES)*, Federal Information Processing Standards Publication, 2001.

5. **NIST Special Publication 800-38D**, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, 2007.

6. **NIST FIPS PUB 180-4**, *Secure Hash Standard (SHS)*, 2015.

7. **OWASP Foundation**, *Mobile Application Security Verification Standard (MASVS) Version 2.0*, 2024.

8. **OWASP Foundation**, *OWASP Top 10 Mobile Risks*, 2024.

9. **Microsoft Corporation**, *The STRIDE Threat Model*, Microsoft Security Development Lifecycle, 2024.

10. **Google Developers**, *Android Keystore System & Hardware-backed security*, Android Open Source Project (AOSP) Documentation, 2024.

11. **Google Developers**, *WindowManager.LayoutParams.FLAG_SECURE*, Android API Reference, 2024.

12. **Signal Messenger LLC**, *The Double Ratchet Algorithm & X25519 Protocol Specification*, 2023.

13. **Daniel J. Bernstein**, *Curve25519: high-speed elliptic-curve cryptography*, Public Key Cryptography - PKC 2006, Lecture Notes in Computer Science, Springer, 2006.

14. **Học viện Kỹ thuật Mật mã**, *Giáo trình Cơ sở Mật mã học và An toàn Hệ thống Thông tin*, NXB Quân đội Nhân dân, 2022.

15. **Thư viện Cryptography Dart**, *Cryptography in Dart & Flutter using libsodium and platform APIs*, pub.dev/packages/cryptography, 2024.

16. **Flutter Open Source Project**, *Flutter Architectural Overview & Platform Channels*, flutter.dev/docs, 2024.

PHỤ LỤC: TỔNG HỢP CÁC CƠ CHẾ BẢO MẬT ĐÃ TRIỂN KHAI

STT	Cơ chế bảo mật	Công nghệ / Thuật toán	Vị trí hiện thực	Test case kiểm chứng	Hiệu quả thực nghiệm
1	Thỏa thuận khóa phiên	ECDH X25519 + HKDF-SHA256	`crypto_service.dart`	`crypto_service_test.dart`	Tốc độ 0,78 ms, miễn nhiễm timing attack
2	Mã hóa đầu cuối (E2EE)	AES-256-GCM (Tag 128-bit)	`crypto_service.dart`, `sos_service.dart`	`sos_test.dart`, `chat_e2ee_test.dart`	0 byte plaintext rò rỉ, phát hiện đảo 1 bit
3	Lưu trữ khóa phần cứng	Android Keystore (TEE/StrongBox)	`secure_storage_service.dart`, `auth_service.dart`	`auth_key_provisioning_test.dart`	Private key không thể trích xuất ra ngoài
4	Xác thực ngoài băng	SHA-256 Key Fingerprinting	`crypto_service.dart`, `auth_service.dart`	`auth_key_provisioning_test.dart`	Triệt tiêu tấn công kẻ đứng giữa (MITM)
5	Phòng chống phát lại	ReplayGuard (Nonce + TTL 30p)	`replay_guard.dart`, `sos_service.dart`	`security_attack_suite_test.dart` (ATK-03)	Nonce trùng từ chối trước giải mã
6	Kiến trúc cứu nạn khẩn cấp	Emergency Bypass (Zero Barrier)	`main.dart`, `sos_countdown_screen.dart`	`emergency_bypass_test.dart`	Cứu hộ không rào cản trong 2 giây
7	Bảo vệ vùng nhạy cảm	Scoped Biometrics (`local_auth`)	`biometric_service.dart`, `main.dart`	`emergency_bypass_test.dart`	Khóa vân tay các tab riêng tư
8	Chống chụp trộm màn hình	Native `FLAG_SECURE`	`MainActivity.kt`, `native_security_service.dart`	`emergency_bypass_test.dart`	Chặn screenshot, che mờ Recent Apps
9	Phát hiện xâm phạm thiết bị	Root Detection (`su` & test-keys)	`MainActivity.kt`, `native_security_service.dart`	`security_attack_suite_test.dart` (ATK-04)	Cảnh báo đỏ khi máy bị root
10	Chống giả mạo GPS	Mock Location Detection	`tamper_detector.dart`, `app_state.d`	`security_attack_suite_test.dart`	Nhận diện GPS giả, ghi vết kiểm toán

			`art`	(ATK-04)	
11	Chia sẻ hành trình an toàn	CSPRNG 6 số + SHA-256 + TTL	`trip_share_service.dart`	`trip_share_test.dart`	Mã dùng 1 lần, hết hạn tự động hủy
12	Minh bạch hóa dữ liệu	Audit Logging	`app_state.dart`, `audit_log_screen.dart`	`widget_test.dart`	Người dùng kiểm tra được ai đã xem vị trí

— HẾT BÁO CÁO —